BSc (Hons) in **Computer Science**

SE3062 : Intelligent Systems

Year 3 · Semester 1 · 2026 Semester 2

Faculty of Computing

Practical 01

> **Objective & Lecture Mapping:** Recall that an intelligent agent acts within a continuous loop: Sensors generate percept sequences, and Actuators execute actions to maximize a Performance measure. Use this sheet to execute practical modifications and incrementally evaluate your design against Lecture 01 and 02 theory (from basic recall to analytical classification).

Part 1: Code Analysis & PEAS Evaluation

Task 0 : Setting up and getting the starter Code

1. Go to the Git Hub Repo. Create a Fork of this Git Repo to your Personal Github Profile. Open it using your favourite IDE and follow the below instruction to complete the practical. The base code is in the "master" brach of the repo.

Task 1.1: The Environment State (__init__)

1. (Remember) List the four components of the PEAS framework discussed in Lecture 01.

P - Performance Measure
E - Environment
A - Actuators
S - Sensors

2. (Understand) Look at the `VisualGridHuntGame.__init__` method. Which specific variables in this code represent the physical "Environment (E)" state?

self.width
self.height
self.walls
self.food_positions
self.opponents

3. (Analyze) Based on the variables initialized (specifically `self.opponents`), classify this baseline environment as "Single-Agent" or "Multi-Agent". Briefly justify your choice.

Multi-Agent Environment.
>Inside the __init__ method, self.opponents is initialised and dynamically populated with adversarial agents. since the environment contains multiple moving entities whose actions directly affect the primary agent's performance, it is classified as a Multi-Agent environment.

Task 1.2: The Perception Subsystem (get_percept)

4. (Remember) Define what a "percept sequence" is according to Lecture 01.

A perception sequence is the complete full history of everything that the agent has ever perceived up to the current point in time. It represents the total chronological sequence of percepts received from the environment through its sensors.

5. (Understand) Which component of the PEAS framework does the `get_percept()` method represent in our code?

Sensors
> The get_percept() method gathers information about the environment's state and feeds it into the agent, mimicking how a biological or robotic sensor inputs environmental data.

6. (Evaluate) Based on the exact dictionary returned by `get_percept()`, is this environment "Fully Observable" or "Partially Observable"? Explain why based on what the agent can and cannot see.

Partially Observable Environment.
> The agent receives localised or specific boolean and coordinate indicators but it does not have a complete real-time global map view of all remaining food items and empty spaces across the entire grid simultaneously. because the agent's sensors provide incomplete or restricted information about the entire environment state at any given moment the environment is classified as partially observable

Task 1.3: Action Execution (execute_action)

7. (Understand) When `execute_action()` deducts points for hitting a wall, which PEAS component is being actively updated?

Performance Measure
> In the code, hitting a wall executes self.score -=5. since score represents the metric evaluating how effectively or rationally the agent is behaving in the environment, modifying it directly updates the performance measure.

8. (Evaluate) Why is it structurally important that this metric evaluates changes in the external environment state (hitting a wall) rather than the agent's internal processing effort?

A performance measure should always evaluate what the agent achieves in the external environment rather than how hard the agent tries internally. it performance were tied to internal effort, an inefficient agent could receive a high score simply by consuming processing power without achieving any real objective in the environment. Evaluating external state changes ensures the agent is rewarded only for achieving useful, goal oriented behaviour

Part 2: Guided Modification & Deep Theory Mapping

> Follow the implementation instructions to inject a new hazard, then answer the questions to map your code changes back to the theoretical nature of the environment.

Step 2.1: Extending the Environment Initialization (__init__)

1. **Implementation Instructions:** Declare a new trap collection attribute (e.g., `self.toxic_traps = set()`) inside `__init__`. Populate it with coordinate tuples safely avoiding position `(0, 0)`, walls, and food.

9. (Understand) If you successfully add `self.toxic_traps` to the environment but intentionally hide this data from the agent's sensors, how does this specifically alter the environment's "Observability" classification?

It reinforces and makes the environment strictly Partially Observable.

even though toxic traps physically exist in the environment's state, hiding them from the agent's sensors means the agent cannot perceive the full state of the environment at any given time. The agent will be unawares of dangerous grid coordinates until it steps on them, confirming that information is hidden and the environment is partially observable.

Step 2.2: Updating the Perception Subsystem (get_percept)

1. **Implementation Instructions:** Locate `get_percept(self)` and add a new boolean sensor key (e.g., `'smells_toxin': tuple(self.agent_pos) in self.toxic_traps`) to the dictionary returned to the agent loop.

10. (Analyze) By adding `'smells_toxin'`, you expand the agent's percept sequence. Explain how this specific sensor helps the agent act with "Rationality" (maximizing expected utility).

Partial Observability
> The agent cannot observe the entire grid's toxic trap locations at once. Through the percept, it only receives information regarding whether its current position (or adjacent cells) contains a toxic trap. This mirrors real-world sensors, which only capture local environmental data rather than full global state information.

Step 2.3: Modifying Action Execution & Visual Rendering (execute_action & draw_grid)

1. **Implementation Instructions:** In `execute_action`, check if the updated `self.agent_pos` intersects with `self.toxic_traps` to decrement `self.score` by 15 points. In `draw_grid`, iterate through traps to render custom purple shapes on the canvas.

11. (Remember) You programmed a severe score penalty for stepping on a trap to avoid metric exploitation. What was the classic "Vacuum World Exploit" example discussed in Lecture 01 that illustrated the danger of faulty metrics?

Performance Measure
> Applying a score deduction (penalty) when the agent steps onto a toxic trap directly modifies the Performance Measure. This negative reward incentivizes the agent to maximize its total score by learning to avoid hazardous positions while reaching its goals.

Finalize and Lock Lab 01 Analytical Evaluation

Lab 01 code analysis and theoretical evaluation complete and verified!

FACULTY OF COMPUTING